Sistema de Gerenciamento de Alunos

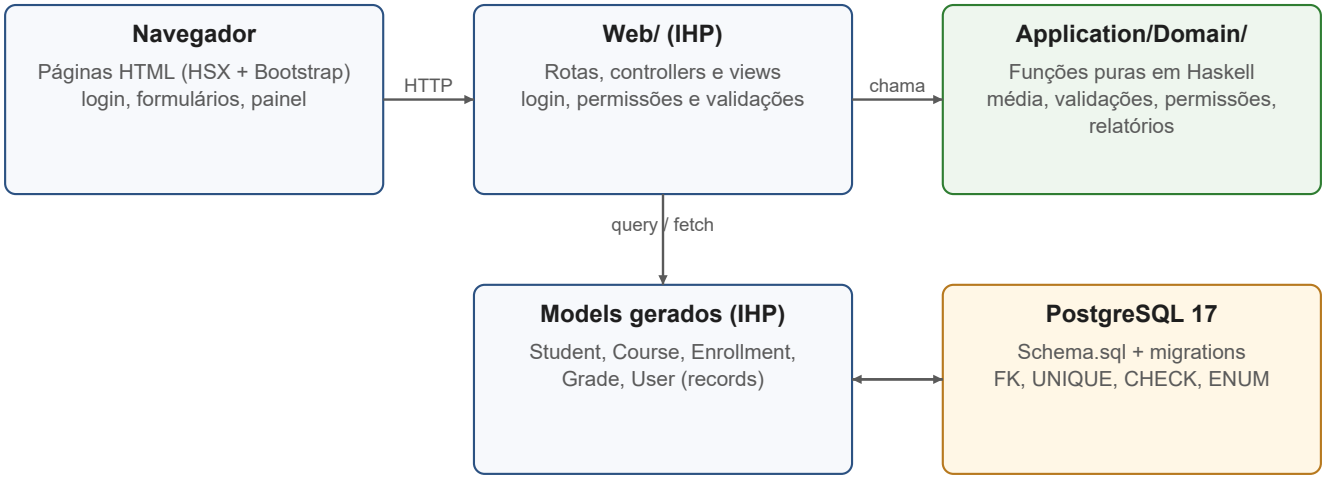
Apresentação do sistema · Haskell + IHP · Programação Funcional

> Aplicação web para cadastrar **alunos, cursos, matrículas e notas**, calcular a **média ponderada** e a **situação** de cada aluno (aprovado, recuperação ou reprovado), com **login por perfil** (administrador e professor) e um **painel de indicadores**. Toda a regra de negócio está em funções puras, separadas do banco e testadas sem IO.

Tecnologias

Camada	Tecnologia
Linguagem e compilador	Haskell · GHC 9.10.3 · Cabal 3.16
Framework web	IHP 1.6 (rotas tipadas, HSX, ORM, geradores de código)
Banco de dados	PostgreSQL 17
Ambiente	Nix (flake) + devenv + direnv, rodando no WSL (Ubuntu)
Testes	Hspec (unitários e de integração) · `nix flake check` · GitHub Actions

Arquitetura

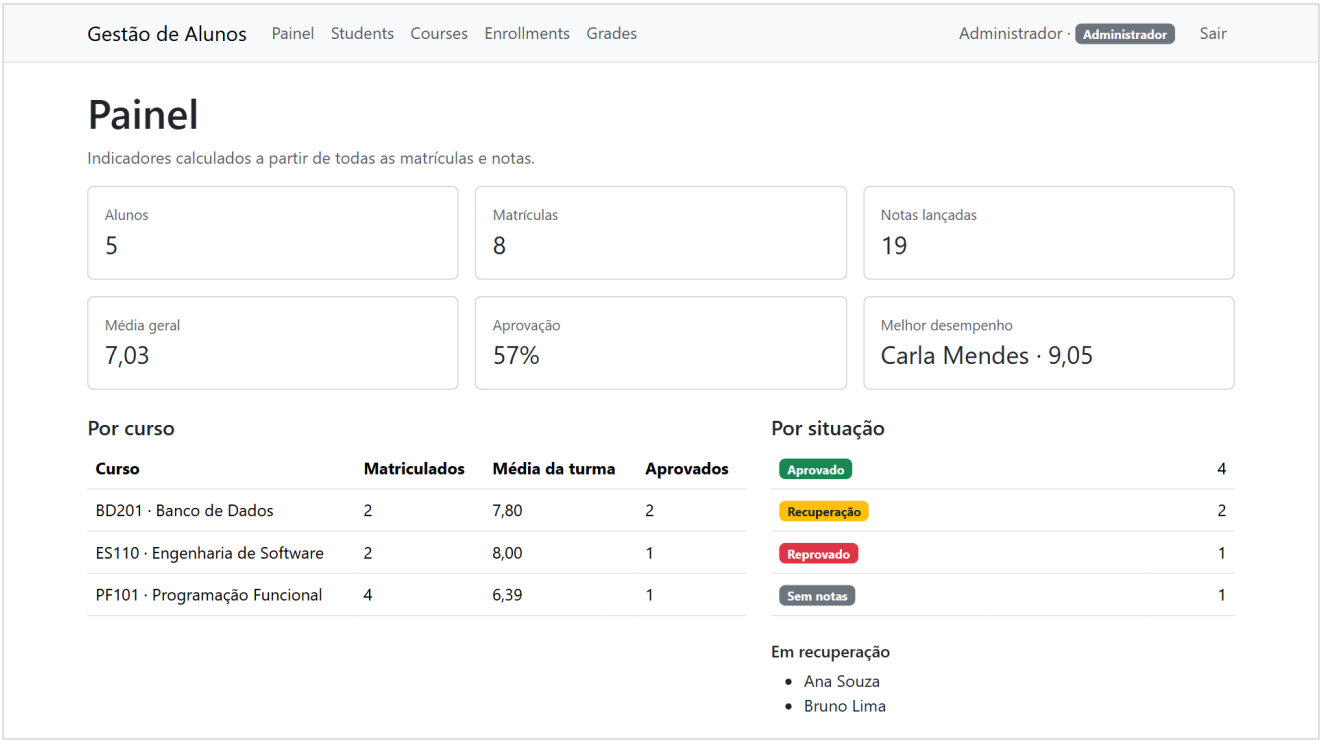


Como rodar

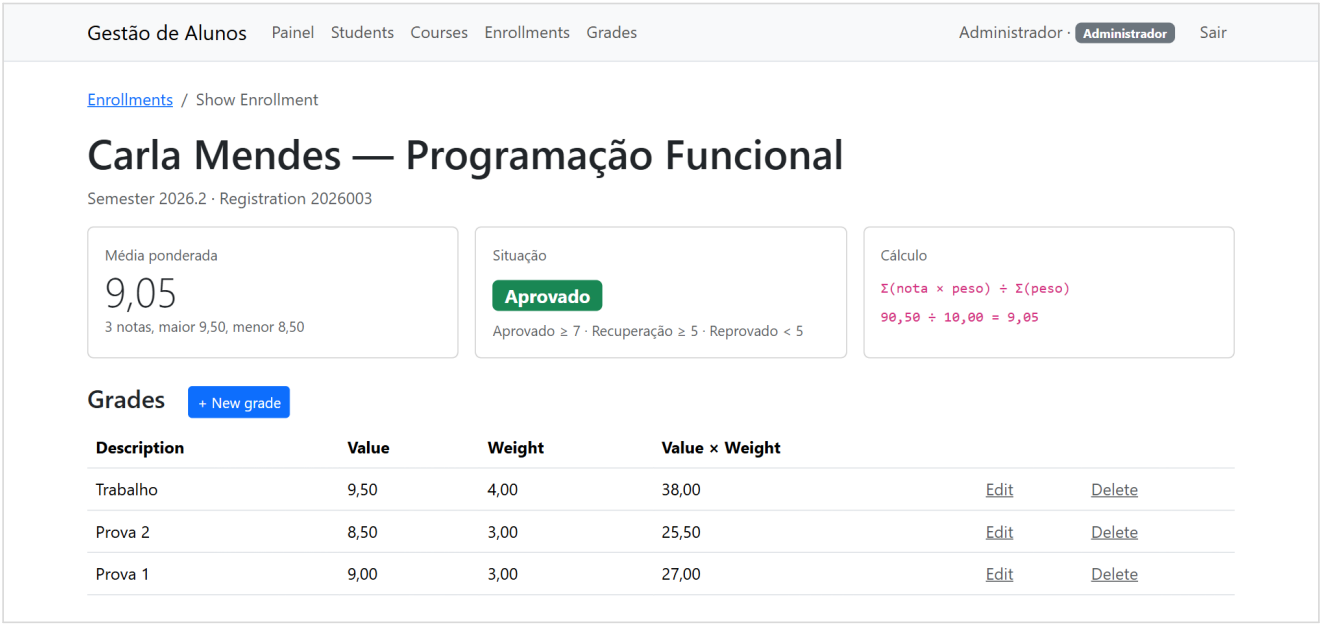
```
cd ~/sistema-gerenciamento-alunos && devenv up     # app em http://localhost:8000
nix flake check --impure                           # testes + build de produção
```

Os usuários de teste (administrador e professor) estão no `README.md`.

O sistema em funcionamento



Painel inicial: indicadores calculados com map, filter, fold e concatMap.



Página da matrícula: notas com peso, conta da média ponderada e situação.

1. Ambiente IHP + Postgres + GHC/Cabal/Nix

O ambiente inteiro é declarado no `flake.nix` e travado no `flake.lock`: quem clona o projeto recebe as mesmas versões de GHC, bibliotecas e Postgres. O `devenv up` sobe o banco e o servidor juntos, e o direnv carrega o ambiente ao entrar na pasta. Roda no WSL porque o Nix não roda nativamente no Windows.

Onde: `flake.nix`, `flake.lock`, `.envrc`, `Makefile`, `Application/Schema.sql`

2. Esqueleto do projeto com scaffolding

Criado com `ihp-new`, segue o padrão MVC do IHP: models gerados a partir do banco, controllers e views em HSX (HTML com checagem de tipos). As rotas são tipos Haskell, então um link para uma ação inexistente nem compila.

Onde: `Main.hs`, `Config/Config.hs`, `Web/Routes.hs`, `Web/Types.hs`, `Web/FrontController.hs`, `Web/View/Layout.hs`

3. Repositório Git + convenções de código

Histórico com **Conventional Commits** (`feat:`, `fix:`), `.gitignore` protegendo o build e o segredo de sessão, formatação definida em `.stylish-haskell.yaml` e CI no GitHub Actions rodando `nix flake check`. Toda mudança de banco é uma **migration** nomeada com o timestamp Unix. Os modelos do banco seguem o IHP em inglês (`Student`), e o domínio fica em português (`Aluno`) para não colidir com os tipos gerados.

Onde: `.git`, `.gitignore`, `.stylish-haskell.yaml`, `.github/workflows/`, `Application/Migration/`, `AGENTS.md`

4. Scaffolds gerados pelo IHP

O comando `new-controller` gerou, para Students, Courses, Enrollments e Grades, o CRUD completo: controller com 7 ações, views Index/New/Show/Edit, bloco de rotas e tipo do controller. Os scaffolds foram testados antes de qualquer mudança (o teste revelou nota 15 aceita e erro 500 em e-mail repetido) e depois ampliados com validações, selects, permissões e relatórios.

Onde: `Web/Controller/*.hs`, Web/View/<Entidade>/, `Web/Routes.hs`

5. Modelos (Student, Course, Enrollment, Grade) com tipos algébricos e records

Duas camadas: os **records do IHP**, gerados do `Schema.sql`, e os **tipos puros do domínio**: records (tipos produto) `Aluno`, `Curso`, `Matricula` e `Avaliacao`, e tipos soma `Situacao` e `ErroValidacao`. O papel do usuário é um ENUM do Postgres que vira o tipo soma `UserRole = Admin | Professor`. `Conversao.hs` traduz uma camada na outra.

Onde: `Application/Domain/Types.hs`, `Application/Domain/Conversao.hs`, `build/Generated/`

```
data Avaliacao = Avaliacao { descricao :: Text, nota :: Double, peso :: Double }
data Situacao  = Aprovado | Recuperacao | Reprovado | SemNotas
```

6. Funções puras de domínio (IMC adaptado para média ponderada, validações)

Como no IMC, que combina duas medidas num índice e o classifica em faixas, a **média ponderada** Σ(nota × peso) ÷ Σ(peso) é classificada em ≥ 7 Aprovado, ≥ 5 Recuperação e < 5 Reprovado. As **validações** devolvem `Either ErroValidacao a` e são as mesmas usadas nos formulários (`validarCom`). Sem IO: 53 testes unitários cobrem o domínio.

Onde: `Application/Domain/Media.hs`, `Application/Domain/Validacao.hs`, `Test/Domain/`

```
termosDaMedia avaliacoes
    | somaDosPesos <= 0 = Nothing
    | otherwise = Just (somaPonderada, somaDosPesos, somaPonderada / somaDosPesos)
  where
    somaPonderada = sum [nota * peso | Avaliacao { nota, peso } <- avaliacoes]
    somaDosPesos  = sum [peso | Avaliacao { peso } <- avaliacoes]
```

7. Closures e currying aplicados

`calculadoraComPesos` recebe os pesos uma vez e devolve uma função que guarda esses pesos (**closure**). `classificar` é `classificarCom` parcialmente aplicada aos cortes 7 e 5 (**currying**); `validarNaoVazio "nome"` vira um validador pronto para o campo nome.

Onde: `Application/Domain/Media.hs`, `Application/Domain/Validacao.hs`

```
mediaProvasETrabalho = calculadoraComPesos [3, 3, 4]   -- closure: lembra os pesos
classificar          = classificarCom 7 5               -- currying / aplicação parcial
```

8. List comprehensions + pattern matching

Comprehensions filtram e transformam numa expressão só, e o padrão à esquerda do `<-` já desmonta o record. Pattern matching aparece nos tipos soma (`descreverSituacao`, `podeAcessar`), no formato das listas (`resumirAvaliacoes [] / [x] / xs@(x:_)`) e na escolha de leitura ou escrita em cada controller.

Onde: `Media.hs`: `boletim`, `alunosPorSituacao`, `contarPorSituacao`, `resumirAvaliacoes`; `Conversao.hs`: `montarMatriculas`

```
alunosPorSituacao situacao matriculas =
    [ nomeAluno | m@Matricula { aluno = Aluno { nomeAluno } } <- matriculas
                , situacaoDaMatricula m == situacao ]
```

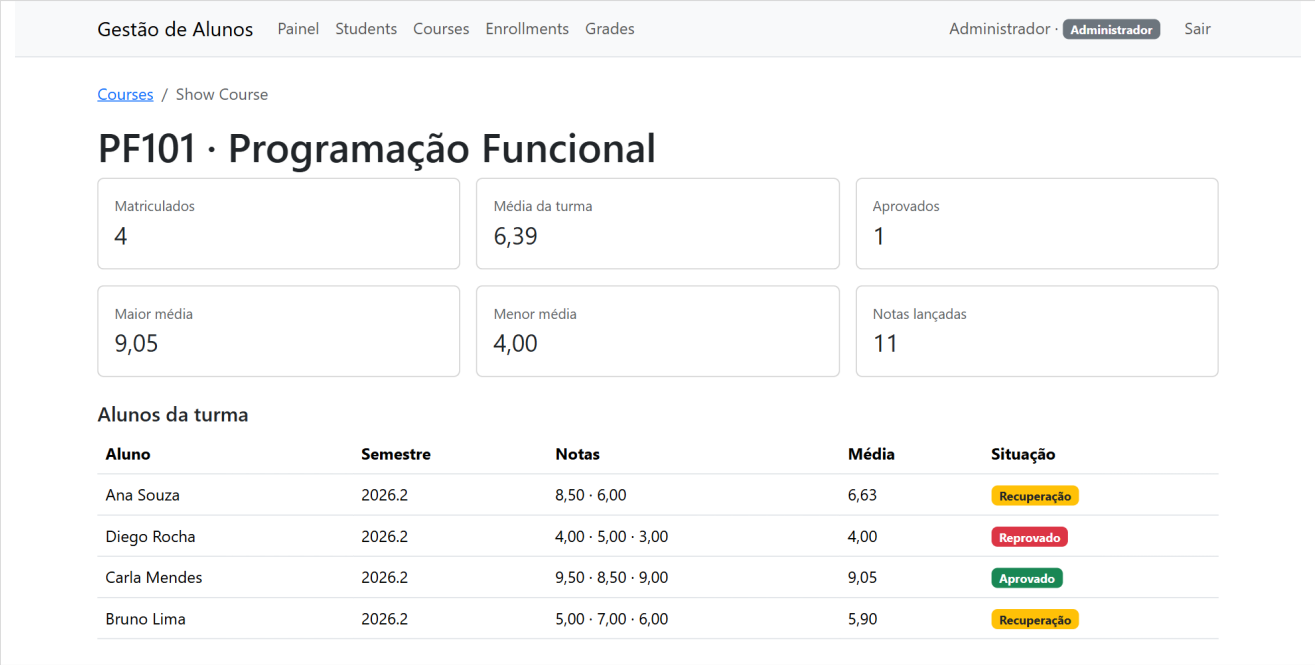
9. Queries com map, filter, fold/reduce e concatMap/flatMap

Os dados vêm do banco com o query builder do IHP (`query @Grade |> fetch`) e são processados por funções puras. Os resultados aparecem no painel, no boletim do aluno e nas estatísticas da turma.

Onde: `Application/Domain/Relatorio.hs`, `Application/Helper/Controller.hs` (`carregarMatriculas`)

Operação	Função	O que faz
`concatMap`	`todasAsAvaliacoes`, `montarMatriculas`	Junta as notas de todas as matrículas; une matrícula + aluno + curso
`map`	`mediasDasMatriculas`, `resumoPorCurso`	Média de cada matrícula; resumo de cada curso
`filter`	`matriculasNaSituacao`, `matriculasDoCurso`	Seleciona por situação, aluno ou curso
`foldl'`	`estatisticas`	Quantidade, soma, maior e menor numa única passada
`foldr`	`taxaDeAprovacao`, `melhorDesempenho`	Conta aprovados; acha a maior média

```
todasAsAvaliacoes = concatMap avaliacoes
estatisticas      = foldl' acumular (Estatisticas 0 0 Nothing Nothing)
mediaGeral        = mediaDasEstatisticas . estatisticas . mediasComNota
```



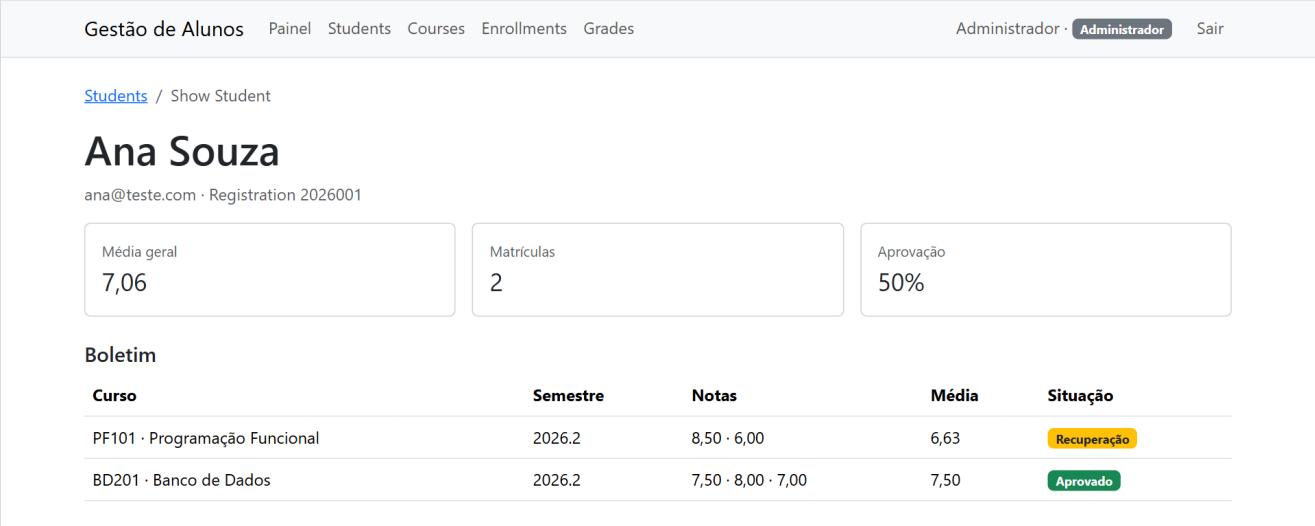
Página do curso: estatísticas da turma (foldl') e lista de alunos (filter).

PARTE 3 · FUNCIONALIDADES

10. CRUD completo de Estudantes e Cursos

Criar, listar, ver, editar e excluir alunos e cursos, gerado pelo IHP e testado operação por operação, conferindo no banco. A página do aluno mostra o **boletim** com média geral; a lista de cursos mostra matriculados e média da turma. Excluir um aluno ou curso remove as matrículas e notas ligadas (`ON DELETE CASCADE`).

Onde: `Web/Controller/Students.hs`, `Web/Controller/Courses.hs`, `Web/View/Students/`, `Web/View/Courses/`



Página do aluno: dados, indicadores e boletim com todas as matrículas.

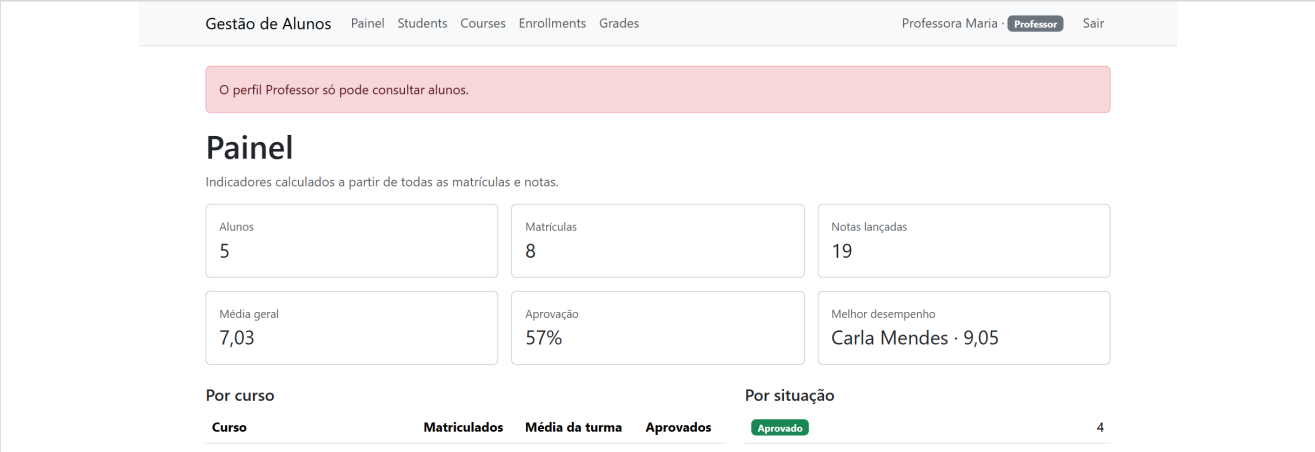
11. Validações de acesso (roles) e restrições de integridade (constraints)

Acesso: login com a autenticação do IHP. A permissão é uma função pura e cada controller a consulta antes de qualquer ação (`beforeAction`). **Administrador** pode tudo; **professor** consulta tudo e só altera notas. Sem login, o usuário vai para a tela de entrada. **Integridade em três níveis:** validação pura no formulário; checagem de duplicidade no banco antes de salvar; e **constraints** no PostgreSQL (FK, UNIQUE, CHECK e ENUM), que recusam dado inválido mesmo fora do app.

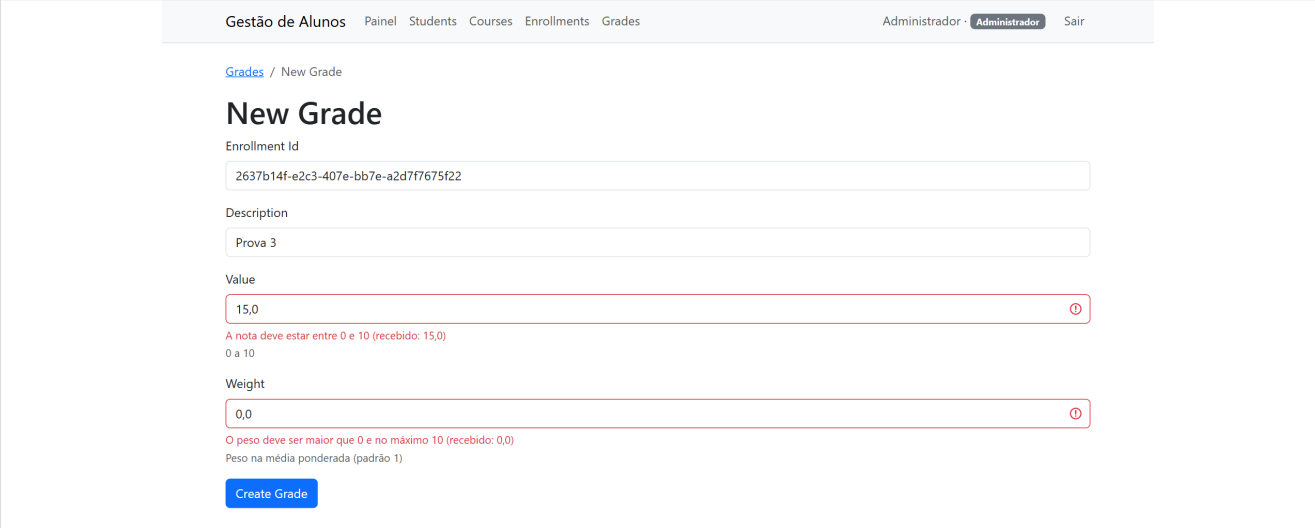
Onde: `Application/Domain/Acesso.hs`, `Web/Controller/Prelude.hs` (`exigirAcesso`), `Application/Schema.sql`, migration `1792072088-...`

```
podeAcessar :: UserRole -> Recurso -> Operacao -> Bool
podeAcessar Admin     _     _        = True
podeAcessar Professor _     Ler      = True
podeAcessar Professor Notas Escrever = True
podeAcessar Professor _     Escrever = False

ALTER TABLE grades ADD CONSTRAINT grades_value_range CHECK ((value >= 0) AND (value <= 10));
```



Professor tentando cadastrar um aluno: a ação é bloqueada e nada é gravado.



Validação no formulário: nota 15 e peso 0 recusados com a mensagem da função pura.

Qualidade e verificação

- **53 testes unitários** (Hspec) das funções puras: médias, validações, permissões e relatórios.
- **Teste de integração** com um banco PostgreSQL temporário.
- `nix flake check --impure` roda os testes e o build de produção; o GitHub Actions roda o mesmo a cada push.
- Testes ponta a ponta pelo HTTP: CRUD, login, permissões por perfil e constraints recusando dados inválidos direto no banco.

Resumo: requisito → onde está

Requisito	Onde está
Ambiente IHP + Postgres + GHC/Cabal/Nix	`flake.nix`, `flake.lock`, `.envrc`
Esqueleto com scaffolding	`Main.hs`, `Config/`, `Web/`
Git + convenções	histórico, `.gitignore`, `.stylish-haskell.yaml`, `.github/`, migrations
Modelos com tipos algébricos e records	`Application/Domain/Types.hs`, `build/Generated/`
Funções puras (média ponderada, validações)	`Media.hs`, `Validacao.hs`
Closures e currying	`calculadoraComPesos`, `classificar`, `validarNaoVazio`
List comprehensions + pattern matching	`Media.hs`, `Conversao.hs`, `Acesso.hs`
Scaffolds gerados pelo IHP	`new-controller` → `Web/Controller/`, `Web/View/`
CRUD de Estudantes e Cursos	`Students.hs`, `Courses.hs` (controllers e views)
Acesso (roles) e constraints	`Acesso.hs`, `exigirAcesso`, `Schema.sql` (CHECK, UNIQUE, FK, ENUM)
map, filter, fold, concatMap	`Application/Domain/Relatorio.hs`